# 1. Introduction

## 1.1 Background and Objective

In satellite attitude determination and control systems (ADCS), the sun sensor plays a vital role in accurately determining the sun vector relative to the satellite body frame. This information is crucial for stabilizing and orienting the spacecraft, especially during sun-pointing or solar array alignment operations. Among various types of sun sensors, the Position Sensitive Detector (PSD)- based sun sensor provides an analog output proportional to the position of the incident light spot on its active surface, enabling precise angular measurement with high resolution and low noise.

The primary objective of this work is to design, interface, and experimentally characterize a PSD-based analog sun sensor similar in architecture to the FS1 model. The aim is to understand its working principle, electrical behavior, and angular response characteristics under controlled laboratory conditions. The developed sensor will later be used for CubeSat-scale ADCS applications or deployable optical telescope systems requiring fine sun-tracking capability.

## 1.2 Role of Sun Sensors in ADCS

Sun sensors are one of the most fundamental attitude sensors used in spacecraft systems. They provide direct measurements of the sun's position relative to the spacecraft body frame, which can be fused with gyroscope and magnetometer data to estimate the complete spacecraft attitude. Depending on accuracy requirements, sun sensors may be used as:

- ***Coarse Sun Sensors (CSS):*** For low-cost, wide field-of-view measurements with limited precision.
- ***Fine Sun Sensors (FSS):*** For high-accuracy applications with narrow field-of-view and sub-degree resolution.

The PSD-based sun sensor discussed here falls in the fine sun sensor category, capable of providing continuous two-axis analog output representing the sun vector with a precision better than one degree when properly illuminated and calibrated.

## 1.3 Scope of the Present Work

The present work focuses on the complete realization of the PSD-based sun sensor system, from understanding the sensor design and working principle to hardware integration, software communication, and performance testing. The key activities undertaken in this work include:

- Studying the structural and electronic design of the FS1 PSD-based sun sensor.
- Interfacing the sensor with a Raspberry Pi 4 via the $I^2C$ communication protocol.
- Developing Python scripts to read raw data, positional outputs, angular values, and temperature.
- Setting up controlled experiments using artificial light sources to analyze the sensor's output stability and response.
- Performing stepper motor–based characterization to evaluate the angular response and repeatability.
- Processing the acquired data using averaging and polynomial curve fitting to assess consistency and accuracy.

The insights obtained through these experiments will be instrumental in improving sensor calibration and enhancing its applicability for future space or CubeSat missions.

# 2. Working Principle of PSD-Based Sun Sensor

## 2.1 Concept and Physical Principle

A Position Sensitive Detector (PSD) registers the position of an incident light spot on a continuous active surface. Physically, when a collimated (or non-collimated) beam falls through an aperture onto the PSD, the light creates a small spot whose location depends on the incident direction of the beam. By measuring the photocurrents (or voltages) at the PSD electrodes, the spot centroid can be inferred and mapped to a sun-vector direction.

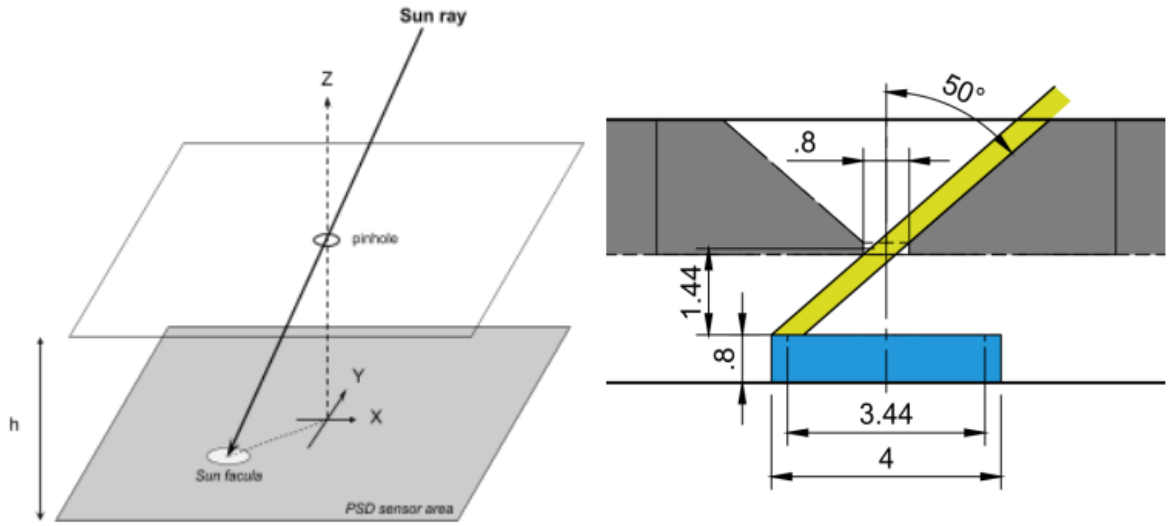



***Figure 2.1***

## 2.2 Onboard Computation (MSP430FR2311)

All primary signal conditioning and geometric computations are performed on the sensor's MCU (MSP430FR2311). The firmware acquires the four corner voltages (Vx1, Vx2, Vy1, Vy2), computes normalized coordinates and applies the calibrated transformations to produce the sun vector and output angles. Key steps performed onboard:

- ADC acquisition and offset/ temperature compensation.
- Normalization to obtain X and Y coordinates:
  $X = (Vx2 + Vy1 - Vx1 - Vy2) / Sum$
  $Y = (Vx2 + Vy2 - Vx1 - Vy1) / Sum$
  where $Sum = Vx1 + Vx2 + Vy1 + Vy2$.
- Conversion from normalized coordinates to incident angles using calibrated relations (linear approximation or polynomial mapping depending on field of view).
- Temperature compensation and packaging of outputs (raw voltages, $X/Y$ position, $\theta x/\theta y$, temperature, status) into I²C-readable registers.

## 2.3 From Spot Position to Sun Vector

The geometric mapping from PSD coordinates to the sun vector uses the aperture-to-PSD distance (effective focal length, f). For small angles, the mapping can be approximated by:

$$\theta x = arctan(X / f) \ , \ \theta y = arctan(Y / f)$$

In practice the MCU implements a calibrated polynomial function $\theta = f_calib(X)$ to compensate for non-linearities, wide field of view, and optical distortions introduced by the aperture and housing. Calibration coefficients are stored in firmware and used in real-time.

## 2.4 SBC: Data Access and Characterization

The Raspberry Pi 4 acts as a host for characterization and validation. It accesses processed outputs via I²C (as defined in the MCU register map) and performs higher-level tasks without altering the sensor's real-time computation. Tasks on the Pi include:

- Periodic I²C reads of registers: raw voltages (optional), $X/Y$, $\theta x/\theta y$, temperature, status.
- Stabilization by software averaging (e.g., block average of 40–50 samples) to reduce short-term fluctuation.
- Logging, plotting, RMSE computation against calibration, and repeatability analysis across multiple runs.

# 3. Sensor Design and Configuration

## 3.1 Structural Overview

The sensor assembly follows the FS1 PSD-based architecture. It comprises an optical entrance (aperture), an optical spacer defining the effective focal length, the PSD active element, front-end analog electronics, and an embedded microcontroller (MSP430FR2311) for signal processing. The design emphasizes mechanical stability, light-tight housing, and thermal considerations to ensure repeatable measurements.

## 3.2 Optical Assembly and Photodiode Layout

The optical path uses an aperture to form a small light spot on the PSD surface. The PSD used in this design provides four output electrodes ($V_{x1}$, $V_{x2}$, $V_{y1}$, $V_{y2}$) placed around the active area. Key optical parameters:

- ***Aperture diameter:*** chosen to balance spot size and field-of-view.
- ***Effective focal length (aperture-to-PSD distance):*** determines sensitivity ($\theta \approx arctan(X/f)$).
- Baffle/blackening inside housing to reduce stray light and reflections.

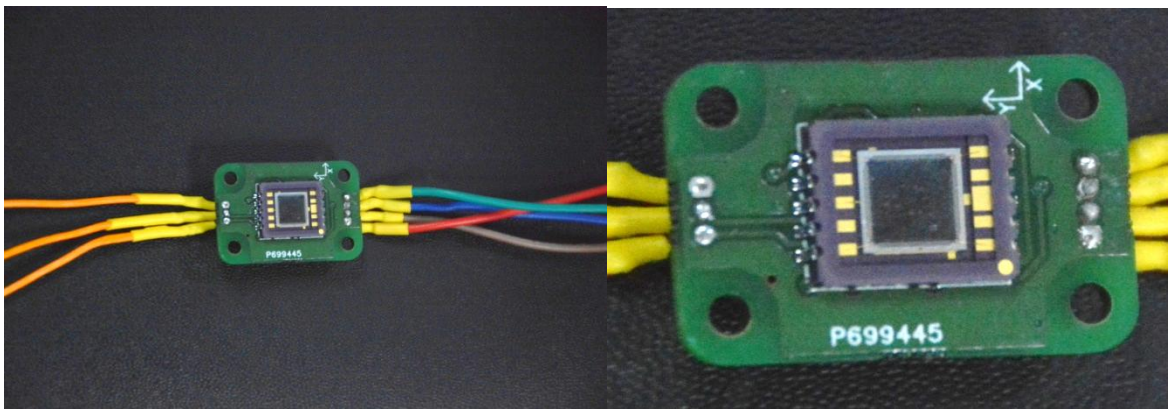



***Figure 3.1 PSD Sun Sensor and Hamamatsu S5990-01 Position Sensitive Device***

## 3.3 Electronics and Signal Conditioning

The analog front-end conditions the low-level photocurrents into measurable voltages and prepares signals for ADC sampling by the MCU. Main elements include:

- Trans-impedance amplifiers (TIA) for each PSD electrode to convert current to voltage.
- Low-pass filtering to reduce high-frequency noise.

- Offset correction and gain calibration circuits (trimmer or digital calibration in firmware).
- Temperature sensor placed near the PSD for compensation in firmware.

The conditioned voltages are sampled by the MSP430FR2311 ADC channels. The MCU performs offset subtraction, normalization, calibration mapping, and packages results for I²C reads.

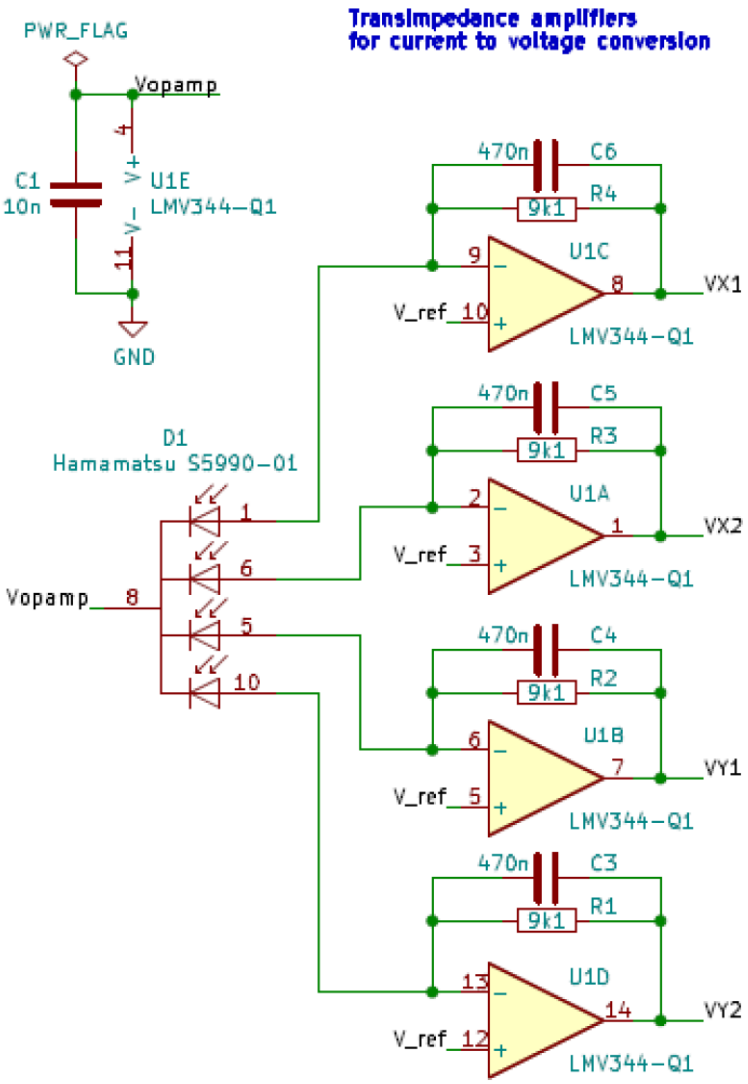



***Figure 3.2 Schematic of Transimpedance Amplifiers***

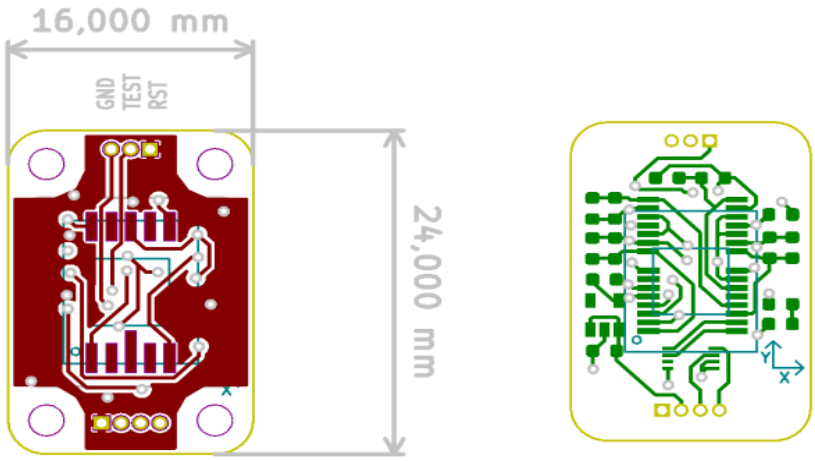



*Figure 3.3 PCB layout: front-side and back-side of hardware*

## 3.4 Embedded MCU and Firmware Responsibilities

The MSP430FR2311 is responsible for real-time processing. Firmware features include:

- Multi-channel ADC acquisition and averaging for each measurement cycle.
- Normalization and polynomial calibration to compute $X/Y$ and $\theta x/\theta y$.
- Temperature compensation and health/status flag generation.
- I²C slave implementation exposing registers for: raw voltages, normalized $X/Y$, computed angles, temperature, and error/status codes.

## 3.5 Mechanical Housing and Mounting

The housing is machined to maintain precise aperture-to-PSD spacing and to provide a rigid mount for integration with test fixtures or spacecraft panels. Features:

- Mounting holes compatible with stepper motor fixture for characterization.
- Internal baffling and matte-black surfaces to minimize internal reflections.
- Thermal path for heat dissipation and a local temperature sensor near the PSD.

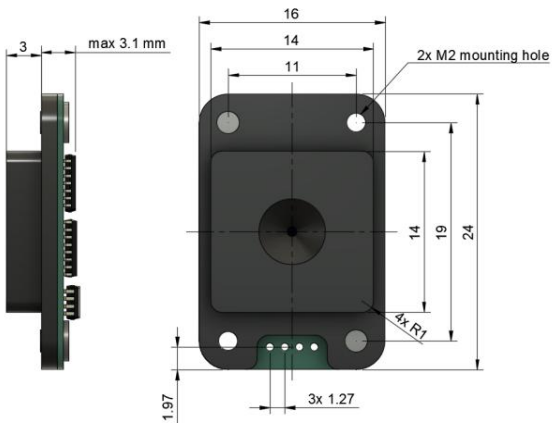



***Figure 3.4 Required panel cut out for the module. The preferred holes for panel mounting are top-right and bottom-left holes.***

## 3.6 Test and Integration Considerations

- Ensure the sensor is light-sealed; even small stray light can affect Sum and hence normalized outputs.
- Provide connector/interface breakout for I²C, power, and optional ADC raw outputs during lab tests.
- Keep mechanical alignment fixtures stable when performing stepper-based angular characterization to ensure repeatability..

# 4. Electrical Interface and Communication

## 4.1 Overview

This sensor uses low-voltage digital communication (I²C) for readout and a small analog front-end (biasing / signal-conditioning) for the PSD outputs. The Raspberry Pi 4 acts as the I²C master and queries the sensor for raw readings (photocurrent/voltage), computed position/angle and temperature.

Key constraints:

- Use **3.3 V logic** on the Pi I²C pins. If the sensor electronics are 5 V, add a proper level shifter.
- Ensure pull-ups on SDA and SCL (typical 4.7 kΩ to 3.3 V) if they are not present on the sensor board.

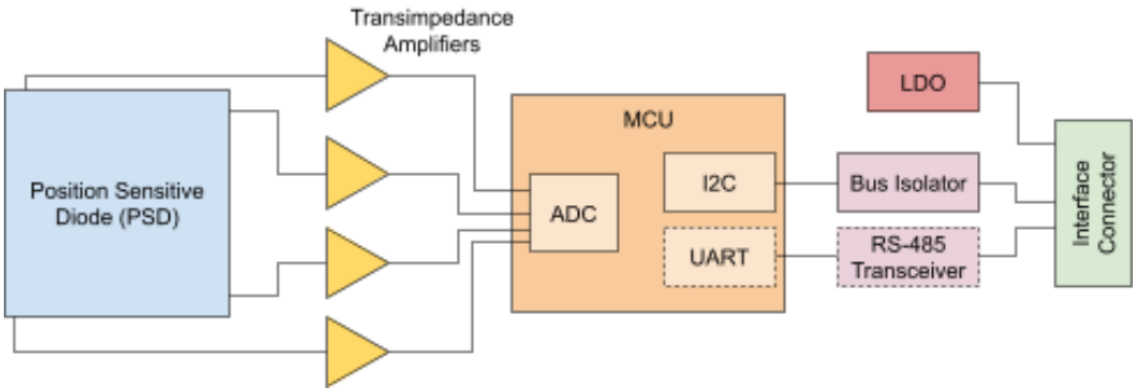



***Figure 4.1 Block diagram of the sensor's electrical components and their connections.***

## 4.2 Power and biasing

- **Supply voltage (VCC):** as specified in design doc (use regulated, low-noise source). If the module expects 5 V internally but exposes 3.3 V I²C, check the board doc.
- **Ground:** single common ground between Pi and sensor.
- **Biasing:** PSD elements typically need a bias/reference circuit. Ensure the reference and bias rails are present and stable before taking measurements.

## 4.3 I²C connection / wiring (Raspberry Pi 4)

Pi pins used:
SDA → physical pin 3 (GPIO2, SDA1)
SCL → physical pin 5 (GPIO3, SCL1)
3.3 V → pin 1 (only if sensor expects 3.3V) or 5V → pin 2 (only if sensor requires 5V and board supports level shifting)
GND → any ground pin
**Pull-ups:** If the sensor board does not include pull-ups, add 4.7 kΩ pull-ups to 3.3 V on SDA and SCL.

## 4.4 I²C addressing and registers (general guidance)

The sensor exposes a small register map over I²C for:
**RAW_x, RAW_y** (raw PSD voltages or ADC counts)
**V_SUM** or **V_TOTAL** (sum or reference)
**ANGLE_X / ANGLE_Y** or computed **azimuth/elevation** (optional, if the module computes angle onboard)
**TEMP** (sensor temperature / internal)
**STATUS / CONFIG** (device status and configuration registers)

## 4.5 Communication reliability tips

- Use `i2cdetect -y 1` (on Pi) to confirm the device address.
- Keep I²C bus length short and avoid noisy power/ground lines.
- Add decoupling capacitors close to the sensor VCC pins.
- For repeated reads, read raw registers in a single multi-byte transaction whenever possible to avoid inconsistent samples between channels.

# 5. Experimental Setup and Methodology

## 5.1 Goals

- Measure sensor stability and noise when pointed at a fixed light source.
- Characterize angular response (single-axis) by rotating the sensor (stepper motor) while keeping source fixed.

## 5.2 Physical setup (static test)

- **Sensor mounting:** sensor fixed on a rigid stand; ensure no mechanical play.
- **Light source:** non-collimated LED/lamp used in initial tests (user note: ideally use a collimated source later). Place source on adjustable stand and align roughly along the sensor normal.
- **Distance:** start close (few cm to tens of cm) then increase distance; record distance for each test.
- **Ambient:** darken ambient light if possible to reduce stray illumination.
- **Photo / schematic placeholder:** (insert photo of sensor + light + stand in final report)

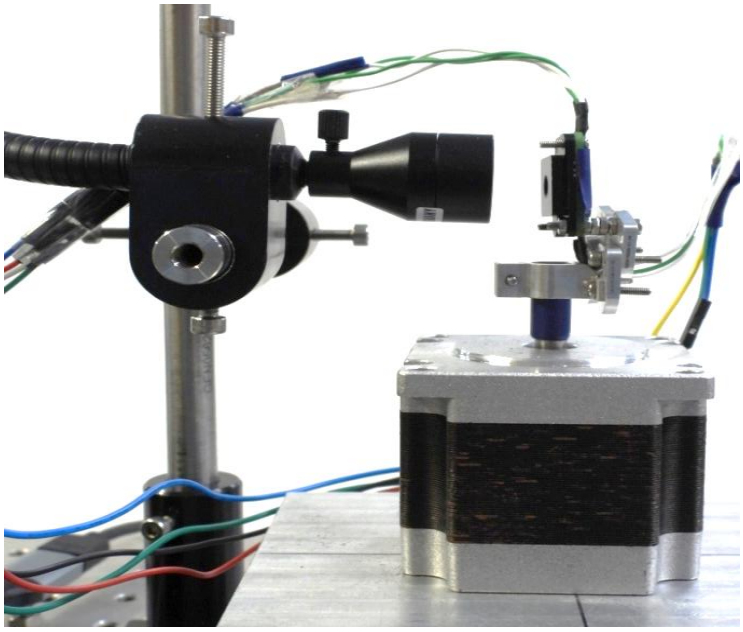


*Figure 5.1 Sun-Sensor fixed on a structure, facing towards the light source*

## 5.3 Single-axis angular characterization

- **Actuation:** mount the sensor on a stepper motor (controlled by a microcontroller) so the sensor rotates about one axis. Do not rotate the light source.

- **Automation:** use a script to command the stepper to step in fixed increments (e.g., 1° or stepper microsteps) and log sensor readings after settling.
- **Settling time:** wait a fixed settling interval after each move (empirically choose, e.g., 0.2–0.5 s) before sampling to avoid transient effects.

## 5.4 Data logging & acquisition

- **Record per-sample:** timestamp, stepper angle (commanded), RAW_X, RAW_Y, V_SUM (if available), computed angle (if present), temperature.
- **Sampling strategy:** for each commanded angle, take **N samples** (e.g., 40) and store mean ± std. Save raw samples for later analysis.
- **File format:** CSV with clear headers (Angle_cmd, mean_X, std_X, mean_Y, std_Y, mean_angle, std_angle, temp, distance, source_intensity).

## 5.5 Calibration / pre-checks

- Verify I²C connection and addresses.
- Warm-up sensor for consistent temperature if internal temperature affects outputs.
- Zero the mechanical mount so commanded stepper angle has known reference.

# 6. Observations and Results *(Static Test)*

## 6.1 Key qualitative observations

- **Distance matters:** when the light source is far from the sensor, readings (angles and raw voltages) showed **large fluctuations** and occasional jumps.
- **Close alignment is stable:** when the source was positioned close and roughly on-axis, the sensor output became **stable** with small fluctuations.
- **Intensity sensitivity:** lowering the light intensity increased noise and fluctuation magnitude.
- **Averaging improves stability:** averaging 40–50 consecutive samples reduced fluctuation to **< 1°** (user observation).

## 6.2 Data Recording

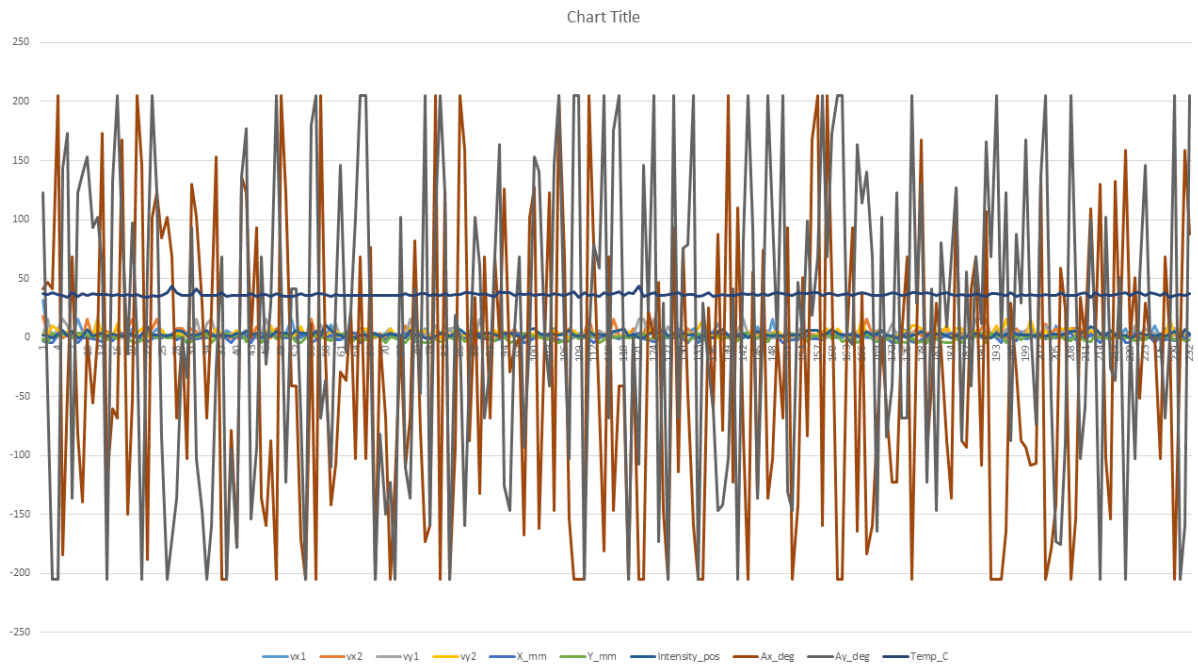



***Figure 6.1 Plot when light source kept at some distance from the sun sensor***

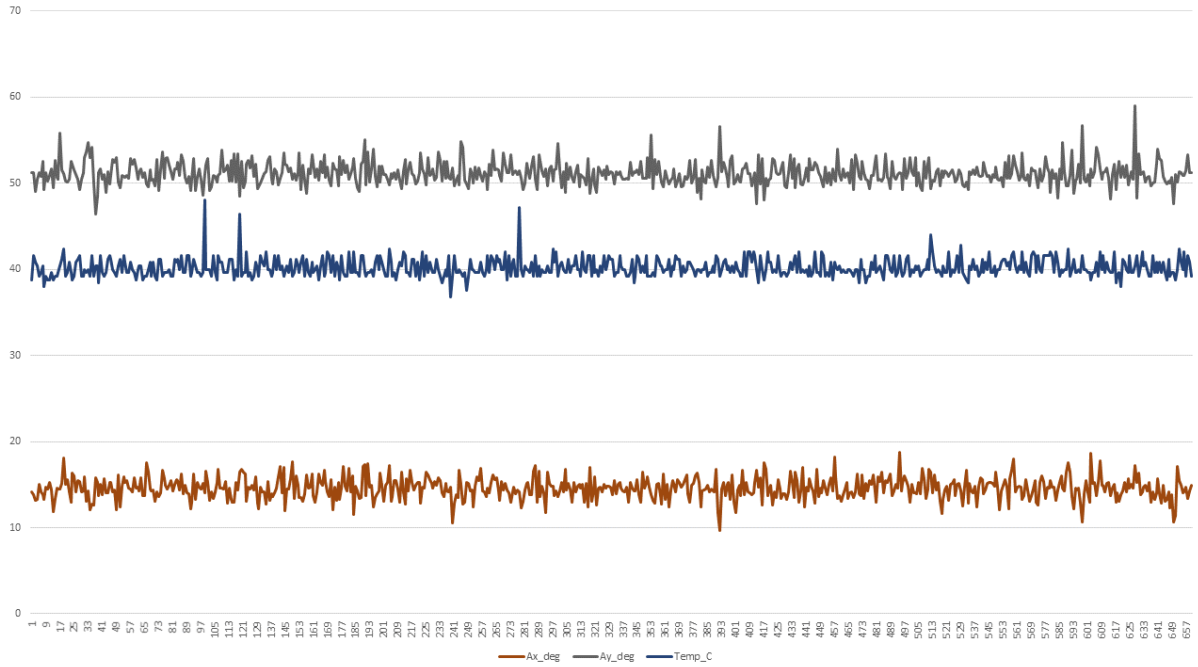



*Figure 6.2 Plot when light source kept too close to sun sensor*

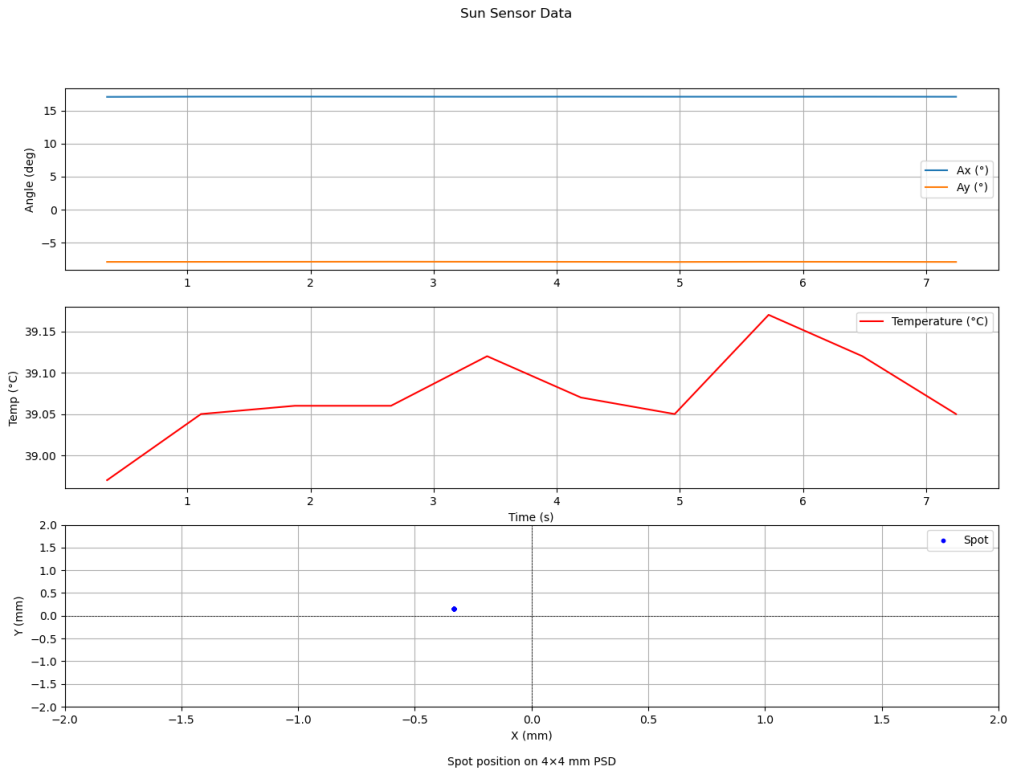



*Figure 6.3 Plot when light source kept too close to sun sensor (averaging the readings)*

## 6.3 Short quantitative result & interpretation

- **Averaging effect:** averaging 40–50 readings reduced fluctuations to **< 1°** — practical for many ADCS test benches.
- **Non-collimated source bias:** when characterizing angular response by rotating the sensor, you observed that a **~50° commanded rotation produced ~100° change** in reported angle. Likely cause: **non-collimated light** (beam divergence changes incident geometry) and possibly nonlinear mapping in the optical assembly.
- **Repeatability is good:** repeated runs showed consistent readings at the same mechanical angle — *repeatability* (good) even if *absolute accuracy* needs a collimated source to confirm.

## 6.4 Recommendations

- Use a **collimated light source** for absolute angular calibration and for verifying linearity (will remove geometric/parallax errors).
- Keep the source **close and well aligned** for low-noise static readings.
- Continue using **n = 40–50 averaging** or implement a lightweight moving-average/low-pass filter in real-time to reduce noise while keeping temporal responsiveness.
- Log raw samples and temperature for each test to separate thermal drift effects from optical/noise effects.

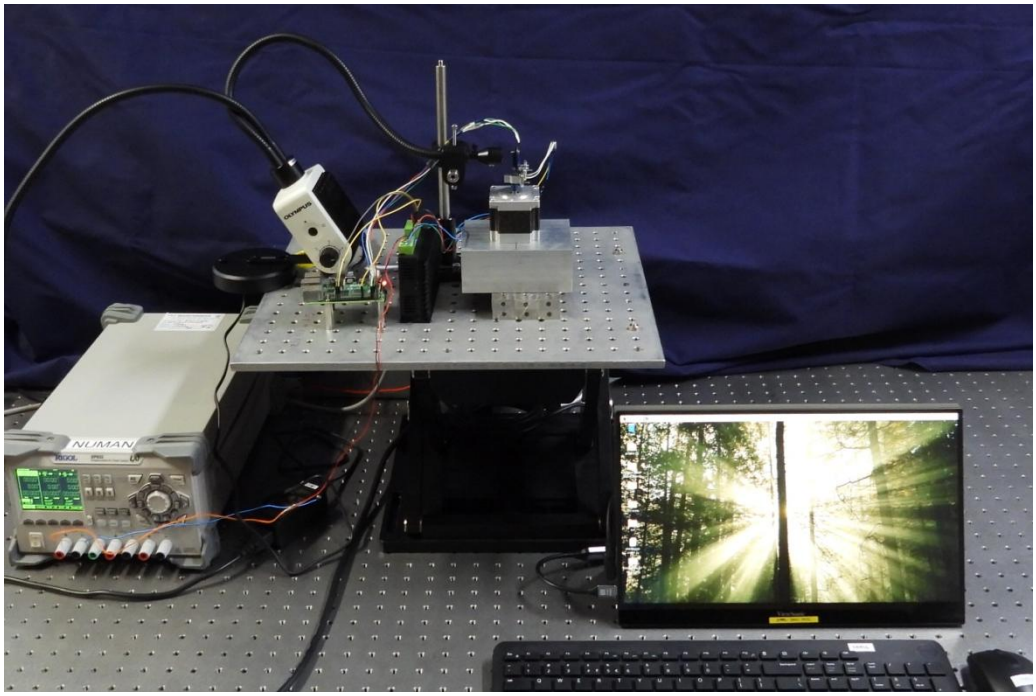
# 7.Experimental Setup *(Angular Characterization)*

## 7.1 Objective

To determine how accurately the sensor responds to changes in incident light angle and to verify consistency and linearity of readings across repeated trials.

## 7.2 Setup Description

- **Mounting:** The sensor was fixed on a **stepper-motor-driven rotation stage** allowing precise angular increments around one axis.
- **Light Source:** A **non-collimated LED source** remained fixed in space. Ideally, a collimated beam would be used to ensure parallel rays, but this test used the available source.
- **Distance:** Source placed at a constant distance from the sensor (as in static test) to maintain illumination uniformity.
- **Alignment:** Zero angle reference was defined when the sensor face was perpendicular to the beam.



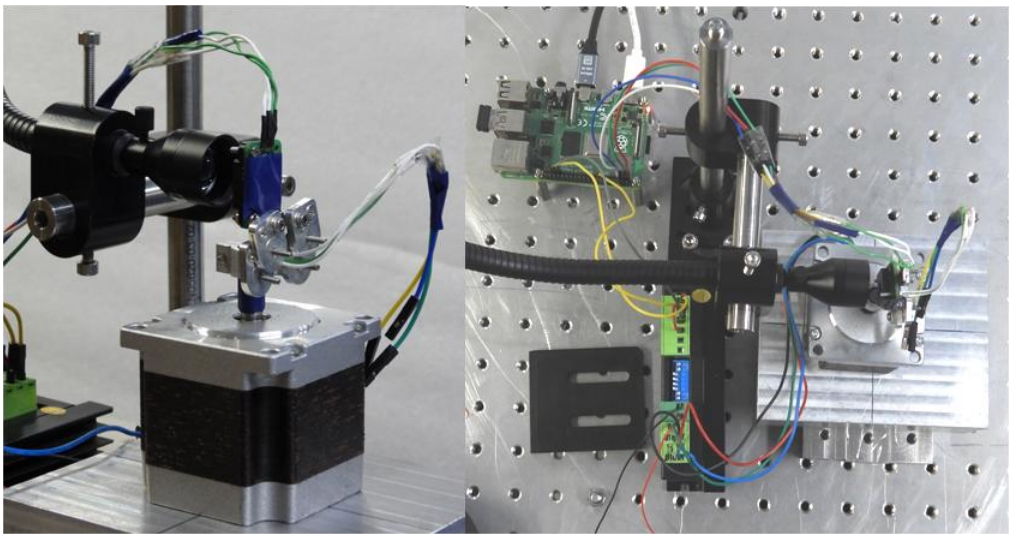


*Figure 7.1 Experimental setup for angular characterization*

## 7.3 Control and Automation

- Stepper motor controlled through a driver interface ( Microstep Driver DM542) commanded from the Raspberry Pi.
- A custom Python script automated rotation, waited for stabilization, and acquired sensor data at each step.
- Each position step collected **40–50 samples**, averaged, and stored with timestamp and commanded mechanical angle.
- Logged parameters: step count, mechanical angle (°), sensor-reported angle (°), raw X/Y readings, temperature, and sample statistics.

## 7.4 Procedure

- Initialize the stepper the with the angle output of sun sensor.
- Rotate the sensor in small increments of 0.225.
- After each rotation, pause for a short delay (≈0.1 s) and acquire readings.
- Continue up to ±50° from center.
- Repeat the entire run for 5 to 10 times to assess repeatability.

## 7.5 Data Recording

Plot of derived angle_sensor vs. stepper_angle and stepper_count to define relationships for comparison across runs.

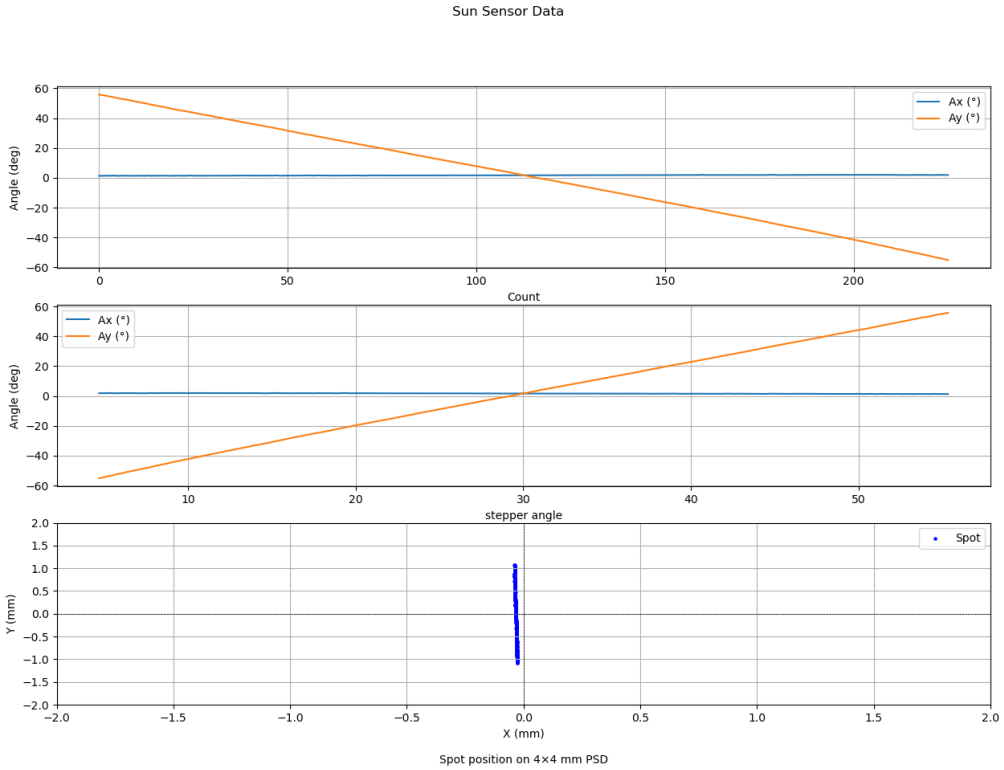



***Figure 7.2 Test 1, plot for sun sensor reading vs actual angle change, when rotating the sensor around its x-axis***

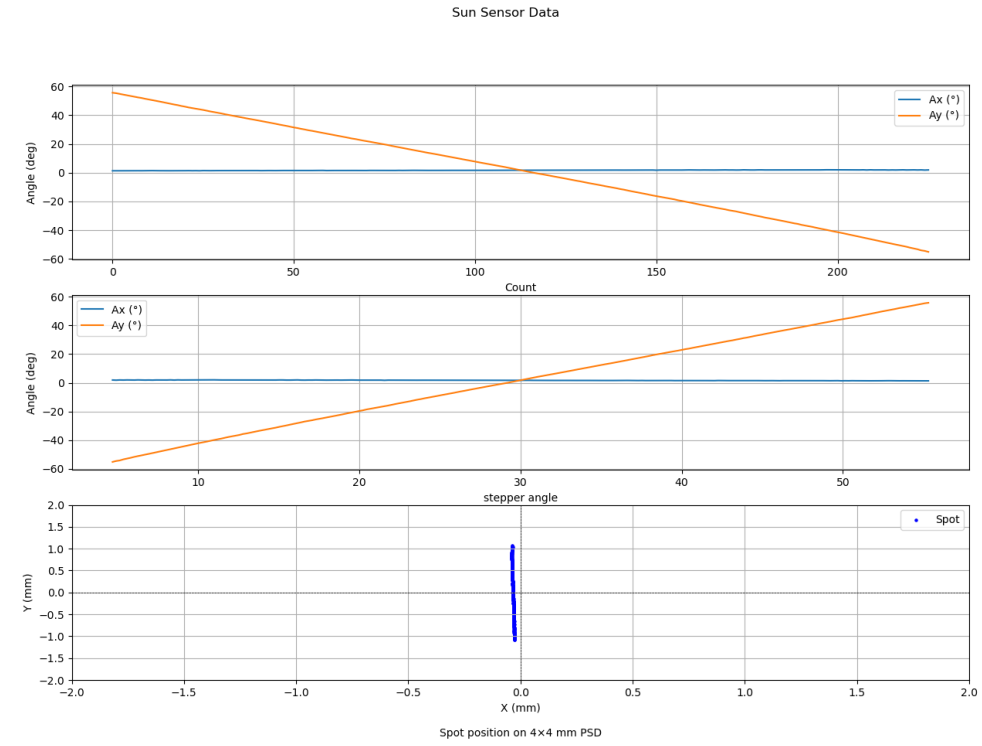



***Figure 7.3 Test 2, plot for sun sensor reading vs actual angle change, when rotating the sensor around its x-axis***

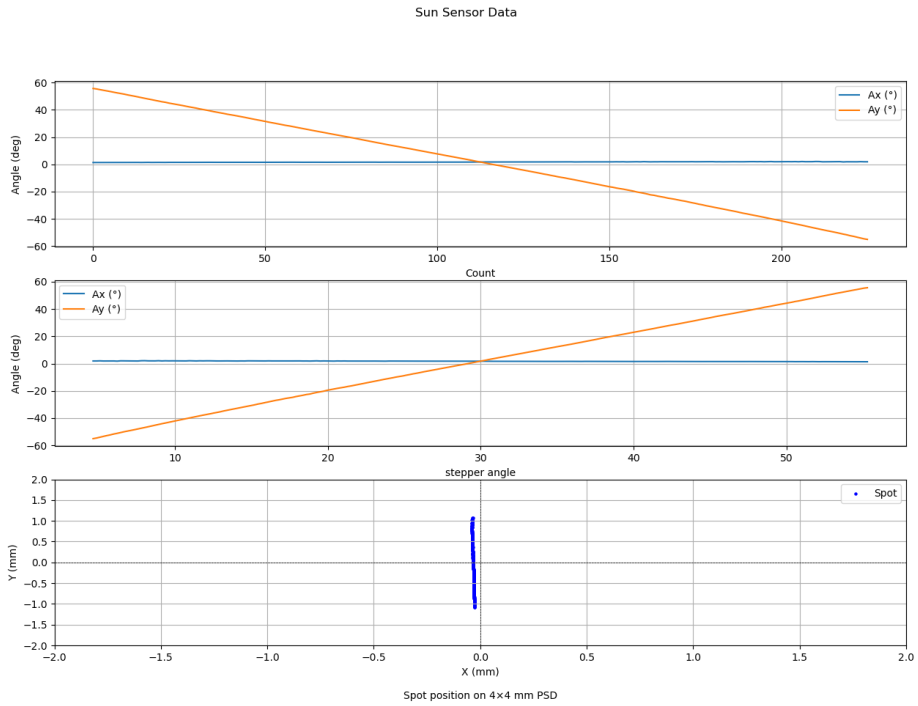



***Figure 7.4 Test 3, plot for sun sensor reading vs actual angle change, when rotating the sensor around its x-axis***

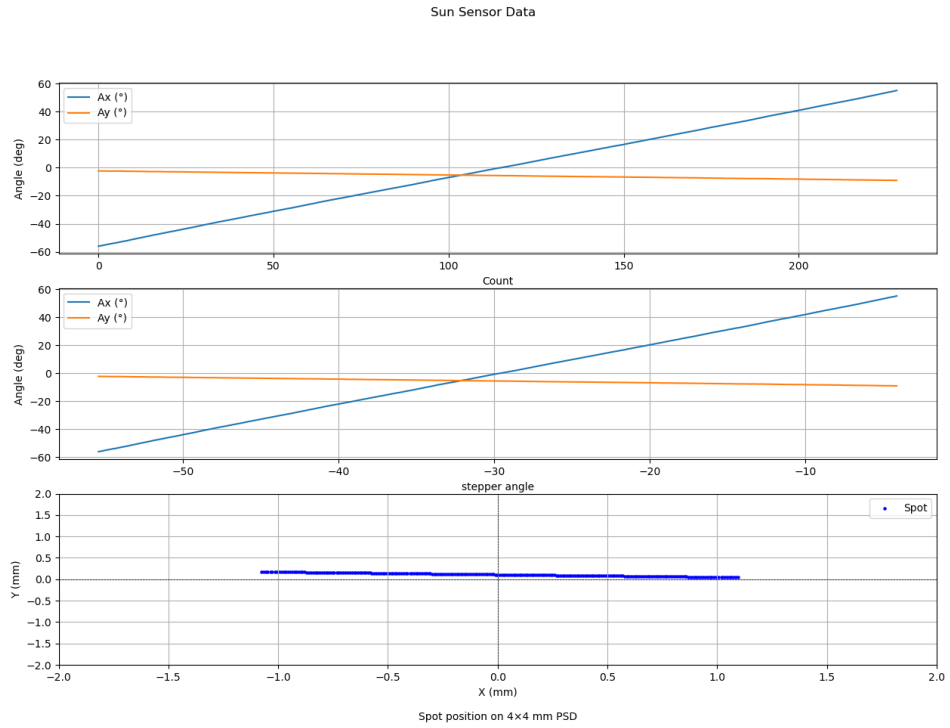



***Figure 7.5 Test 1, plot for sun sensor reading vs actual angle change, when rotating the sensor around its y-axis***

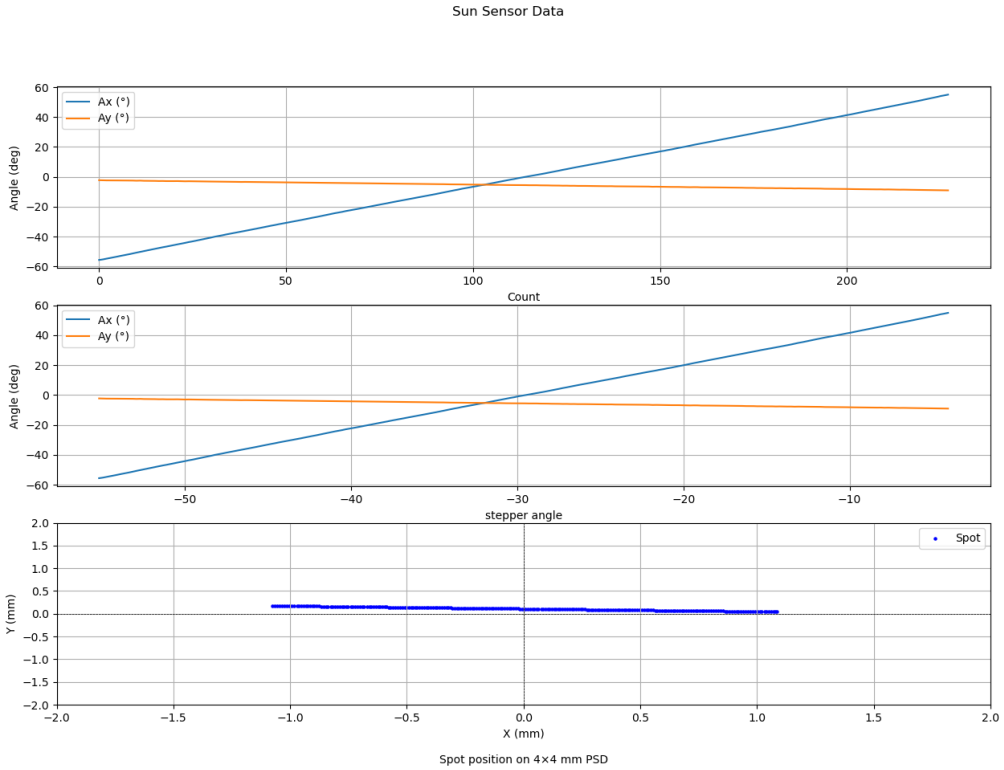



***Figure 7.6 Test 2, plot for sun sensor reading vs actual angle change, when rotating the sensor around its y-axis***

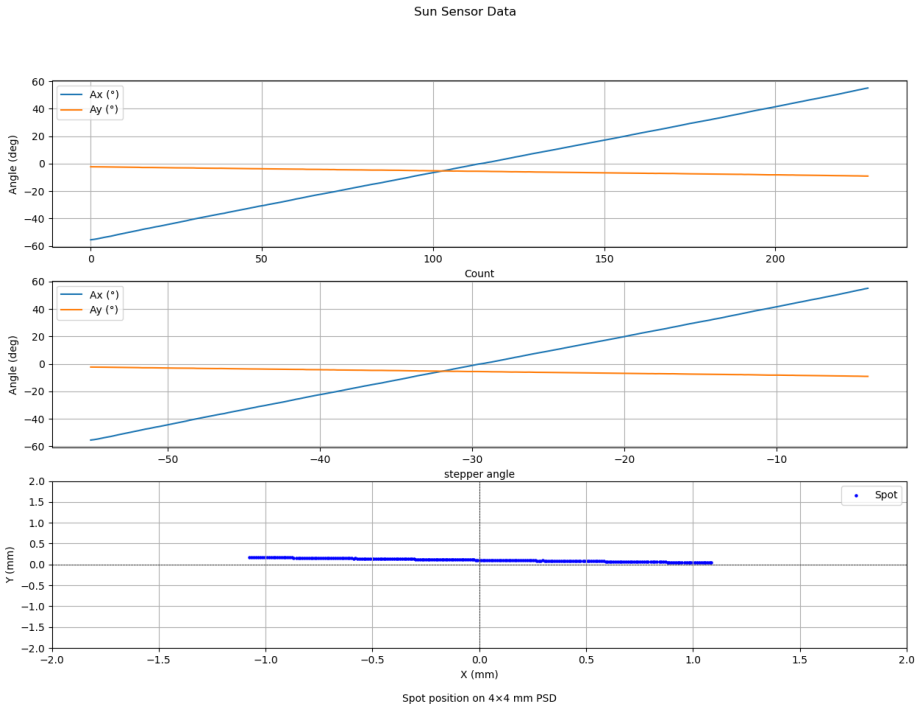



***Figure 7.7 Test 3, plot for sun sensor reading vs actual angle change, when rotating the sensor around its y-axis***

# 8. Analysis and Discussion

## 8.1 Behaviour of Angular Response

- A roughly **linear trend** was observed between mechanical angle and sensor-reported angle; however, the slope exceeded unity.
- A **~50° mechanical rotation corresponded to ~100° sensor output change**, indicating optical magnification from the non-collimated source geometry.
- The response curve remained **consistent** across repeated trials, confirming good repeatability of the sensor electronics and signal chain.

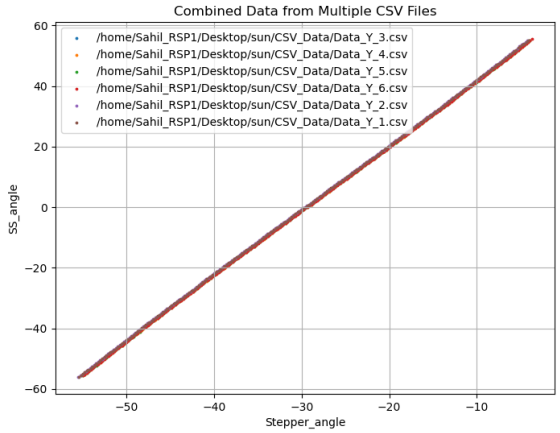



***Figure 8.1 sun-sensor angle output vs actual stepper angle change curve obtained from multiple trials in y-axis***

## 8.2 Repeatability and Stability

- For each repeated test, the same mechanical orientation produced nearly identical readings (variation < 1–2°).
- This demonstrates **excellent consistency**, even though the absolute scaling needs optical calibration.
- Temperature drift had negligible impact over the short test duration (< 0.2 °C variation).

## 8.3 Polynomial Fitting and RMSE Evaluation

Measured data fitted with low-order polynomial models ($2^{nd}$ and $4^{th}$ order) to capture non-linear mapping between mechanical and sensor angles. The ***Root Mean Square Error (RMSE)*** of fits across trials provided a quantitative measure of stability.

```
For rotation in RY ..................................

Equation:
0.002007 x + 2.282 x^2 - 64.8
Equation:
0.002042 x + 2.285 x^2 - 64.63
Equation:
0.002002 x + 2.281 x^2 – 65

RMSE
0.29036748416447855
0.2927186951921902
0.29411706231510537

For rotation in RX ....................................

Equation:
0.0002371 x + 2.131 x^2 + 63.01
Equation:
0.0002167 x + 2.132 x^2 + 62.79
Equation:
0.0002326 x + 2.131 x^2 + 62.79


RMSE
0.24655423860162393
0.24735034908376333
0.2522516355964574
```

## 8.4 Discussion of Sources of Error

- **Non-collimated light:** Major contributor to angular exaggeration; divergence causes unequal illumination across PSD surface.
- **Alignment errors:** Manual mounting and imperfect axis alignment add small systematic offsets.
- **Optical crosstalk:** Scattering within sensor housing can distort edge response.
- **ADC noise / timing jitter:** Negligible after averaging 40–50 samples.

## 8.5 Interpretation

- Despite the scaling bias, the **consistency** of sensor readings suggests the internal circuitry and PSD response are functioning correctly.
- The characteristic curve can be used to **calibrate** true incident angles through polynomial mapping.

Once a **collimated beam** is used, the slope should approach unity, validating the true angle measurement accuracy.

## 8.6 Future Steps

- Repeat the same angular characterization with a **collimated light source** to verify linear mapping.
- Extend to **two-axis rotation** to map full sensor field-of-view.
- Implement **real-time calibration** in the Python acquisition code to auto-convert raw data to corrected angles.
- Use the polynomial with lowest RMSE for on-board computation in the final ADCS module.

# 10. Reference

P. Niemelä, PSD Sun Sensor Design Document (Foresail-1), Aalto University, Department of Electronics and Nanoengineering, Jan. 26, 2023.

# 11. Appendix

## 4.6 Example Python (smbus2):  read & average loop

```
# Example: read consecutive RAW_X, RAW_Y, TEMP registers and average
from smbus2 import SMBus
import time
import statistics

I2C_BUS = 1
I2C_ADDR = 0x4A  # replace with actual 7-bit address from design doc
REG_RAW_X = 0x10  # placeholder register addresses; use FS1 doc values
REG_RAW_Y = 0x12
REG_TEMP  = 0x14

def read16(bus, addr, reg):
    # read two bytes, big or little endian per doc
    data = bus.read_i2c_block_data(addr, reg, 2)
    return data[0] << 8 | data[1]

def sample_once(bus):
    x = read16(bus, I2C_ADDR, REG_RAW_X)
    y = read16(bus, I2C_ADDR, REG_RAW_Y)
    t = read16(bus, I2C_ADDR, REG_TEMP)
    return x, y, t
```

```
def averaged_sample(n=50, delay=0.005):
    with SMBus(I2C_BUS) as bus:
        xs, ys, ts = [], [], []
        for _ in range(n):
            x,y,t = sample_once(bus)
            xs.append(x); ys.append(y); ts.append(t)
            time.sleep(delay)
    return statistics.mean(xs), statistics.mean(ys), statistics.mean(ts), statistics.pstdev(xs), statistics.pstdev(ys)


if __name__ == "__main__":
    mean_x, mean_y, mean_t, std_x, std_y = averaged_sample(n=40)
    print("X:", mean_x, "±", std_x)
    print("Y:", mean_y, "±", std_y)
    print("Temp:", mean_t)
```

Averaging: user found 40–50 samples gives <1° fluctuation, use `n=40` as a good starting point.